Universidad Autónoma de Nuevo León

Facultad de Ciencias Físico Matemáticas

Programación Orientada a Objetos

Manual Técnico: Sistema Administrativo

Nombres:

Melina Sánchez Ávila 2120543

Jorge Castillo Acosta 2132842

Lugar: Monterrey, NL. Fecha 24/10/2025

CONTENIDO

1.- Introducción 2
2. Descripción general del sistema 3
3. Requerimientos técnicos 4
Requisitos de Hardware 4
Requisitos de Software 5
Conectividad 5
Librerías, Frameworks y APIs empleadas 5
4.- Diseño del Sistema 7
Diagrama UML Completo 13
Estructura general de las tablas 14
Diagrama Entidad Relación 16
Script de BD (SQL Script) 17
5.- Instalación técnica 24
6.- Detalle de módulos y funciones 27
Método LeerNumeroempleado 27
Metodo exportar tabla 28
Limpiar Campos 29
7.- Consideraciones finales 30

1.- INTRODUCCIÓN

El presente Manual Técnico del Sistema Administrativo tiene como objetivo ofrecer una exposición detallada, precisa y estructurada de los aspectos técnicos que conforman el funcionamiento interno del programa. Este documento busca servir como una referencia integral para comprender la arquitectura del sistema, su diseño modular, los componentes principales, y la forma en que estos interactúan entre sí para garantizar el correcto desempeño de las operaciones administrativas.

El Sistema Administrativo ha sido desarrollado utilizando principalmente el lenguaje de programación Java, debido a su robustez, portabilidad y la facilidad que ofrece para implementar los principios de la Programación Orientada a Objetos (POO). Este paradigma permitió estructurar el sistema de manera modular y escalable, dividiendo las distintas funcionalidades en clases y paquetes bien definidos. Cada ventana de trabajo, componente gráfico, botón, campo de entrada, lista desplegable y formulario ha sido diseñado con un enfoque orientado a objetos, lo que facilita la reutilización del código y su mantenimiento a largo plazo.

El documento también describe el modo en que el programa interactúa con la base de datos, elemento esencial para el almacenamiento persistente de la información del personal de la empresa. Se detallan los mecanismos de conexión, envío y recepción de peticiones (requests) hacia la base de datos, así como las operaciones de inserción, actualización, eliminación y consulta de registros. Estas operaciones garantizan la integridad y consistencia de los datos administrativos que se manejan dentro del sistema, además de permitir su recuperación o modificación de manera controlada.

Este manual ha sido elaborado con el propósito específico de servir como guía técnica para desarrolladores, ingenieros de software, personal de mantenimiento, administradores de sistemas y analistas técnicos que requieran conocer a fondo la estructura y funcionamiento interno del programa. A diferencia del manual de usuario, este documento hace uso de terminología especializada, explicaciones de bajo nivel y referencias directas a los frameworks, librerías y herramientas utilizadas durante el desarrollo del sistema. Por ello, se asume que el lector cuenta con conocimientos técnicos previos en programación, gestión de bases de datos y entornos de desarrollo integrados (IDE) como IntelliJ IDEA o NetBeans.

Asimismo, este documento incluye diagramas de arquitectura, diagramas UML de clases y secuencia, descripciones de los módulos funcionales, dependencias externas y las rutas lógicas que el sistema sigue durante la ejecución de sus procesos. Se explican también las configuraciones de entorno, los parámetros de conexión a la base de datos, las rutas de los recursos y la estructura de archivos que conforman el proyecto, con el fin de facilitar tareas de mantenimiento, depuración o futuras ampliaciones del sistema.

El alcance de este Manual Técnico abarca todos los componentes esenciales del Sistema Administrativo, desde la capa de presentación (interfaz gráfica) hasta la capa de acceso a datos y la lógica de negocio. Incluye información sobre la estructura de carpetas, la organización del código fuente, la configuración de la base de datos, los requerimientos técnicos de instalación y los procedimientos de despliegue en entornos locales o de producción.

Este manual no tiene como objetivo instruir al usuario final en el uso cotidiano del sistema —esa función corresponde al Manual de Usuario— sino ofrecer a los profesionales de TI la información necesaria para comprender, modificar, depurar o extender las funcionalidades existentes.

2. DESCRIPCIÓN GENERAL DEL SISTEMA

El **Sistema Administrativo** tiene como objetivo principal la **gestión integral de las operaciones de venta de la empresa**, permitiendo almacenar, consultar, modificar y eliminar información relacionada con los productos, insumos, proveedores, clientes y transacciones comerciales de manera estructurada y eficiente. Este sistema constituye una herramienta fundamental para mantener un **control centralizado y seguro de los procesos de venta**, contribuyendo a la automatización de tareas administrativas, al seguimiento detallado del inventario y al registro confiable de cada operación comercial. Asimismo, el sistema incluye la administración de los **datos de los empleados encargados de las ventas**, de modo que sea posible vincular las transacciones a cada vendedor, calcular comisiones y generar reportes individuales de desempeño. De esta manera, el sistema proporciona una plataforma integral que optimiza la gestión de ventas y fortalece la trazabilidad de toda la información comercial dentro de la empresa.

Su propósito general es facilitar la administración del capital humano mediante una **base de datos bien organizada**, capaz de registrar los datos esenciales de cada empleado, tales como nombre, CURP, RFC, puesto, teléfono, correo electrónico y dirección, entre otros. A su vez, el sistema fue concebido con una visión **escalable y extensible**, lo que le permite adaptarse a nuevas necesidades o integrarse en el futuro con otros módulos de gestión empresarial, como nóminas, inventarios o control de ventas.

Desde el punto de vista del desarrollo, el programa fue construido bajo el **paradigma de la Programación Orientada a Objetos (POO)**, lo cual permitió diseñar una estructura modular, reutilizable y mantenible. Cada **ventana, formulario y componente gráfico** del sistema está sustentado en clases base que definen el comportamiento común y facilitan la creación de nuevas interfaces a partir de la misma estructura. De esta manera, la interfaz de usuario mantiene coherencia visual y funcional en todas sus secciones, al mismo tiempo que simplifica el proceso de mantenimiento y ampliación del código fuente.

En cuanto a su **arquitectura general**, el sistema está compuesto por un conjunto de clases principales que controlan la ejecución y las operaciones del programa. Entre ellas destacan el archivo **"Main"**, encargado de iniciar la aplicación y establecer el flujo inicial del programa, y las clases **"W_Administrar_Empleados"**, **"W_Empleados"** y **"W_Listado_Empleados"**, que representan las ventanas y módulos de interacción más importantes. Estas clases gestionan las operaciones de registro, visualización y edición de datos del personal.

Además, el sistema incluye otras clases complementarias con funciones específicas, tales como **"Conexión"**, **"LimpiezaEmpleados"**, y otras utilitarias.

- La clase **"Conexión"** es una de las piezas más críticas del sistema, pues se encarga de establecer y mantener el vínculo con la **base de datos MySQL** instalada localmente o en un servidor. A través de esta clase se ejecutan las operaciones de envío y recepción de datos, utilizando instrucciones SQL encapsuladas dentro de métodos especializados que garantizan la seguridad, consistencia y eficiencia en la comunicación con la base de datos.

- La clase **"LimpiezaEmpleados"** tiene la función de realizar operaciones de depuración, validación de entradas o eliminación controlada de registros, asegurando la integridad de los datos.

- Otras clases secundarias proporcionan soporte adicional para la gestión de eventos, la carga dinámica de recursos gráficos, y la aplicación de estilos visuales a las ventanas mediante librerías externas especializadas en interfaces de usuario.

Todos estos archivos están organizados dentro del **paquete principal del proyecto Java**, siguiendo una estructura jerárquica que facilita su comprensión y mantenimiento. Los recursos gráficos (iconos, etiquetas, fondos, botones personalizados, etc.) provienen de **librerías externas de GUI** (como Swing o JavaFX, dependiendo de la versión utilizada), que permiten un diseño visual más amigable e interactivo.

En el ámbito de almacenamiento, el **subsistema de base de datos** se implementó en **MySQL**, aprovechando su estabilidad, soporte multiplataforma y compatibilidad con Java a través del conector JDBC. La base de datos se compone de múltiples tablas, diseñadas para soportar tanto la gestión actual de empleados como posibles expansiones futuras del sistema. Entre ellas destacan:

- **"empleados"**: tabla principal del sistema, encargada de almacenar los registros completos del personal activo e histórico de la empresa.

- **"artículos"**, **"ventas"** y **"venta_detalle"**: estructuras preparadas para posibles módulos complementarios que se incluirán en versiones futuras del sistema, orientadas a la administración de inventario y operaciones comerciales.

Por ahora, la tabla **"empleados"** constituye el núcleo operativo de esta versión, y todas las funciones principales del sistema —como el alta, la baja y la consulta de registros— se apoyan directamente en ella. Cada campo de la tabla fue diseñado con tipos de datos específicos para optimizar el almacenamiento y garantizar la consistencia de la información.

En términos generales, el **Sistema Administrativo** combina una arquitectura lógica sólida, un diseño modular orientado a objetos y un sistema de base de datos relacional confiable, logrando así una solución estable, escalable y preparada para evolucionar conforme a las necesidades administrativas de la organización.

3. REQUERIMIENTOS TÉCNICOS

REQUISITOS DE HARDWARE

- Procesador: Intel Pentium o superiores.
- Memoria RAM: 4GB o superior.
- Disco duro: por lo menos 500MB de espacio mínimo disponible para la instalación del programa y al menos 1GB de espacio disponible para guardar los registros de la base de datos y el uso continuo del sistema.
- Periféricos: al menos una pantalla para la visualización, un teclado y mouse para acceder e ingresar información al programa.

REQUISITOS DE SOFTWARE

- SO: Preferentemente algún sistema Windows 10 u 11.
- Se recomienda tener instalado Java y algún software para ejecutar programas basados en Java, así como MySQL y un software que se capaz de ejecutar y crear bases de datos en este lenguaje.

CONECTIVIDAD

El programa requiere hacer conexión con la base de datos local que debe crear el equipo técnico al momento de instalarlo dentro del equipo. La conexión a la base de datos es automática y solo requiere de una ligera configuración en el código fuente por el momento.

Además, para ciertas funciones claves de operación se requiere conexión a internet permanente para el correcto funcionamiento del programa.

LIBRERÍAS, FRAMEWORKS Y APIS EMPLEADAS

El **Sistema Administrativo** fue desarrollado empleando diversas librerías y APIs que permiten una implementación robusta, eficiente y modular de las funciones que lo conforman. A continuación, se describen las más relevantes según su propósito dentro del sistema:

1. **Java SE (Standard Edition) API**

La base fundamental del sistema se apoya en la API estándar de **Java SE**, la cual provee las clases esenciales para la estructura y ejecución del programa.

Se utilizan ampliamente los paquetes:

- java.lang – para manejo de tipos primitivos y excepciones.
- java.util – para colecciones, listas, iteradores y estructuras de datos.
- java.io – para entrada y salida de archivos.
- java.sql – para la comunicación con la base de datos.
- javax.swing o javafx.* – para la creación de las interfaces gráficas (según la tecnología elegida).

Estas librerías forman la columna vertebral del sistema, permitiendo la creación de clases, métodos, ventanas y eventos de manera estructurada y coherente con los principios de la Programación Orientada a Objetos.

2. **Swing / JavaFX (Interfaz gráfica de usuario)**

El sistema implementa componentes visuales mediante la biblioteca **Swing** (o alternativamente **JavaFX**, según la versión del desarrollo). Estas librerías son responsables de la creación y administración de los elementos gráficos de la interfaz, como botones, campos de texto, menús, paneles, etiquetas e iconos.

Su uso permite diseñar un entorno visual amigable, dinámico y consistente, con la posibilidad de manejar eventos de usuario (clics, validaciones, acciones, etc.) mediante controladores integrados en las clases de las ventanas.

3. **JDBC (Java Database Connectivity)**

La comunicación entre el sistema y la base de datos **MySQL** se realiza a través de la API **JDBC (Java Database Connectivity)**.

Esta API proporciona las clases necesarias para establecer la conexión con la base de datos, ejecutar consultas SQL, recuperar resultados y gestionar transacciones.

En este proyecto, la clase **"Conexión.java"** utiliza el **driver JDBC de MySQL**, comúnmente importado como:

```
import java.sql.Connection;

import java.sql.DriverManager;
```

De esta forma, el sistema puede enviar y recibir datos de manera segura y eficiente entre la aplicación Java y el servidor MySQL.

4. **MySQL Connector/J**

Para que la API JDBC pueda interactuar con MySQL, el sistema requiere el **conector oficial de MySQL para Java**, conocido como **MySQL Connector/J**. Este archivo .jar permite que el sistema establezca comunicación directa con el servidor de base de datos, autenticando usuarios y gestionando las consultas SQL. Su inclusión es fundamental para garantizar la persistencia de los datos y la estabilidad del sistema en entornos locales o en red.

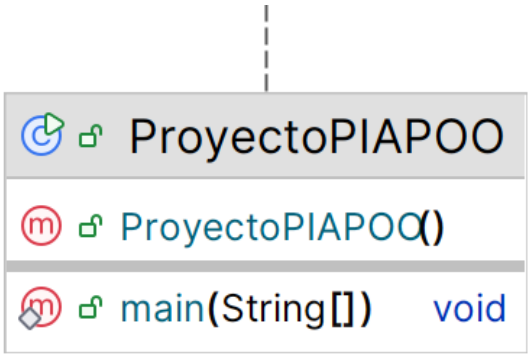
5. **Dipomex API**

Además, el sistema incorpora la integración con la **API de Dipomex**, un servicio en línea que permite obtener información geográfica y administrativa relacionada con los **códigos postales de México**. Esta API facilita la obtención automática de datos complementarios de dirección, como la colonia, municipio, estado y localidad, a partir del código postal ingresado por el usuario en el formulario de registro. Su implementación dentro del **Sistema Administrativo** mejora la precisión y rapidez en la captura de direcciones, reduciendo errores humanos y estandarizando la información almacenada en la base de datos. El consumo de esta API se realiza mediante solicitudes HTTP y el procesamiento de respuestas en formato **JSON**, lo cual garantiza compatibilidad con la arquitectura Java y una integración eficiente dentro del flujo general del sistema.

4.- DISEÑO DEL SISTEMA

El diseño técnico del Sistema Administrativo se basa en un enfoque estructurado y modular, sustentado en los principios de la Programación Orientada a Objetos (POO). El sistema está compuesto por un conjunto de clases que colaboran entre sí mediante relaciones de asociación, herencia y dependencia, con el objetivo de lograr una arquitectura coherente, reutilizable y fácilmente mantenible. La estructura general del sistema puede observarse en el **diagrama UML adjunto**, donde se representan las clases principales, sus atributos, métodos y las interacciones entre los distintos módulos.

En la parte superior de la jerarquía se encuentra la clase **ProyectoPAPOO**, que actúa como punto de entrada de la aplicación y contiene el método main(String[] args). Desde esta clase se inicializa la ventana principal del sistema y se establece la secuencia de ejecución que enlaza con el resto de los módulos funcionales. Su responsabilidad principal es iniciar el entorno gráfico, conectar los controladores y preparar la comunicación con la base de datos.



1 Clase ProyectoPIAPOO

El componente **Principal** representa el núcleo visual de la aplicación. Esta clase contiene el menú principal del sistema, construido mediante componentes JMenuBar, JMenu y JMenuItem. Desde esta ventana se accede a las distintas secciones del sistema, como la administración de empleados, la visualización de listados o el mantenimiento general. Además, la clase gestiona eventos de usuario asociados a cada opción del menú mediante métodos ActionPerformed, los cuales permiten abrir las ventanas secundarias (W_Administrar_Empleados, W_Empleados, W_Listado_Empleados, entre otras) y establecer la conexión con los demás módulos del sistema.

2 Clase Principal

En la capa de interacción con el usuario se encuentran las clases **W_Administrar_Empleados**, **W_Empleados** y **W_Listado_Empleados**, que conforman el conjunto de interfaces gráficas destinadas a la gestión directa del personal.

- **W_Administrar_Empleados** controla las operaciones de alta, baja, búsqueda y modificación de registros. Contiene componentes como botones, campos de texto y listas desplegables, además de métodos que validan la información antes de enviarla a la base de datos.

3 Clase W_Administrar_Empleados

- **W_Empleados** encapsula los datos individuales de cada empleado, actuando como una interfaz de edición detallada. Su diseño permite manipular información personal y laboral, incluyendo CURP, RFC, puesto, teléfono, y estatus.

W_Empleados	
W_Empleados()	
CURP	JTextField
Busca_Cp	JButton
CB_Colonia	JComboBox<String>
Sexo	JComboBox<String>
jLabel7	JLabel
jLabel19	JLabel
jLabel22	JLabel
jLabel17	JLabel
jLabel6	JLabel
Nom_Empleado	JTextField
jLabel23	JLabel
jLabel10	JLabel
Id_Empleado	JTextField
enlace	Conexion
B_Salir	JToggleButton
E_Mail	JTextField
jLabel25	JLabel
Telefono_2	JTextField
jLabel14	JLabel
jLabel16	JLabel
jLabel9	JLabel
jLabel8	JLabel
Telefono_1	JTextField
jPanel2	JPanel
jLabel2	JLabel
CB_Ciudad	JComboBox<String>
jLabel21	JLabel
CP	JTextField
jLabel12	JLabel
RFC	JTextField
jLabel1	JLabel
jPanel3	JPanel
jLabel13	JLabel
jLabel15	JLabel
jLabel4	JLabel
jLabel24	JLabel
conectado	Connection
jLabel11	JLabel
Fecha_Contrato	JDateChooser
Fec_Nac	JDateChooser
Calle	JTextField
limpieza	LimpiezaEmpleados
jLabel20	JLabel
CB_Estado	JComboBox<String>
jLabel5	JLabel
B_Grabar	JButton
Comision	JSpinner
jLabel3	JLabel
No_Ext	JTextField
jLabel18	JLabel
Nom_EmpleadoFocusLost(FocusEvent)	void
Telefono_1KeyTyped(KeyEvent)	void
Nom_EmpleadoActionPerformed(ActionEvent)	void
CPKeyTyped(KeyEvent)	void
limpiacombox(JComboBox<String>, JComboBox<String>, JComboBox<St	
formInternalFrameClosed(InternalFrameEvent)	void
initComponents ()	void
getTextField()	Object
No_ExtKeyTyped(KeyEvent)	void
Telefono_2KeyTyped(KeyEvent)	void
B_GrabarActionPerformed(ActionEvent)	void
InicializaCampos()	void
LeerNumEmpleado()	int
B_SalirActionPerformed(ActionEvent)	void
Busca_CpActionPerformed(ActionEvent)	void
No_ExtActionPerformed(ActionEvent)	void

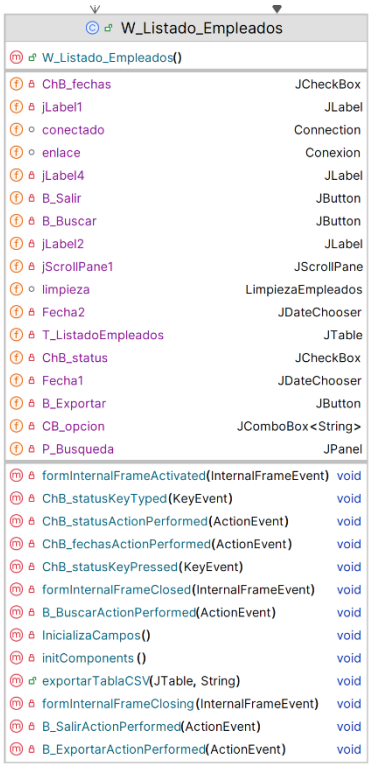
5 Clase W_Empleados

- **W_Listado_Empleados**, por su parte, muestra de manera tabular los registros existentes en la base de datos, permitiendo búsquedas filtradas, actualizaciones rápidas y exportación de información.

Estas clases están estrechamente vinculadas entre sí, manteniendo relaciones de composición y dependencia con los módulos de limpieza, conexión y validación de datos.

La clase Conexión constituye un componente crítico dentro de la arquitectura, ya que gestiona la comunicación directa con la base de datos MySQL. En ella se implementan los métodos necesarios para abrir, mantener y cerrar la conexión a través del driver JDBC (Java Database Connectivity). Otras clases utilizan sus servicios para enviar consultas SQL, recibir resultados y actualizar la información de manera sincronizada. Este diseño permite una separación clara entre la lógica de presentación y la capa de acceso a datos, siguiendo el patrón Modelo–Vista–Controlador (MVC) de manera simplificada.

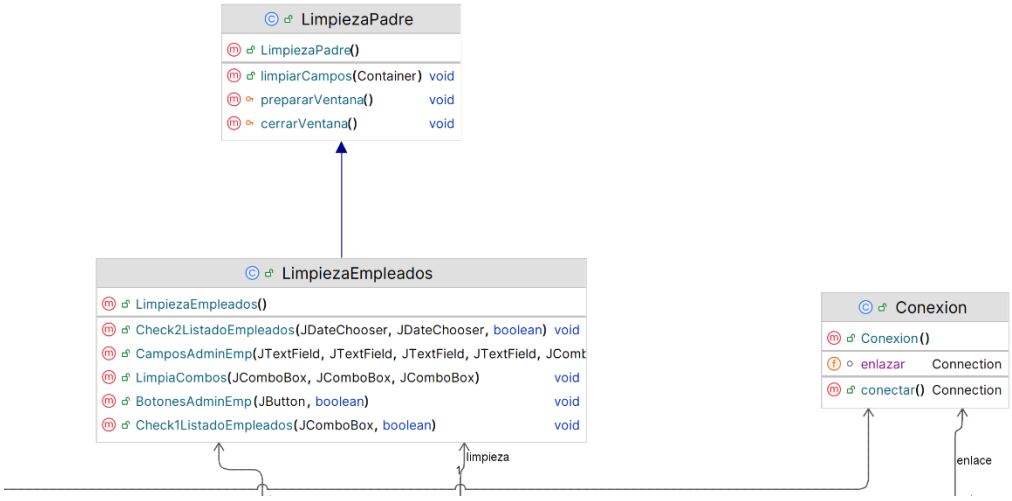
En cuanto a la lógica de limpieza y validación, el sistema emplea una jerarquía basada en las clases **LimpiezaPadre** y **LimpiezaEmpleados**.



6 Clase W_Listado_Empleados

- **LimpiezaPadre** actúa como clase base abstracta que define métodos genéricos de inicialización y cierre de ventanas (inicializarVentana, cerrarVentana), así como la preparación de variables comunes para la interfaz gráfica.

- **LimpiezaEmpleados** hereda de esta clase y especializa sus funciones para el contexto de la gestión de personal, implementando métodos como CheckListaEmpleado(), BotonesAdminEmp(), y CamposAdminLimpiar(). Estos procedimientos se encargan de limpiar formularios, validar campos vacíos y asegurar que los datos ingresados por el usuario cumplan con las reglas de formato establecidas antes de enviarse a la base de datos.



7 Clases Limpieza y Conexión

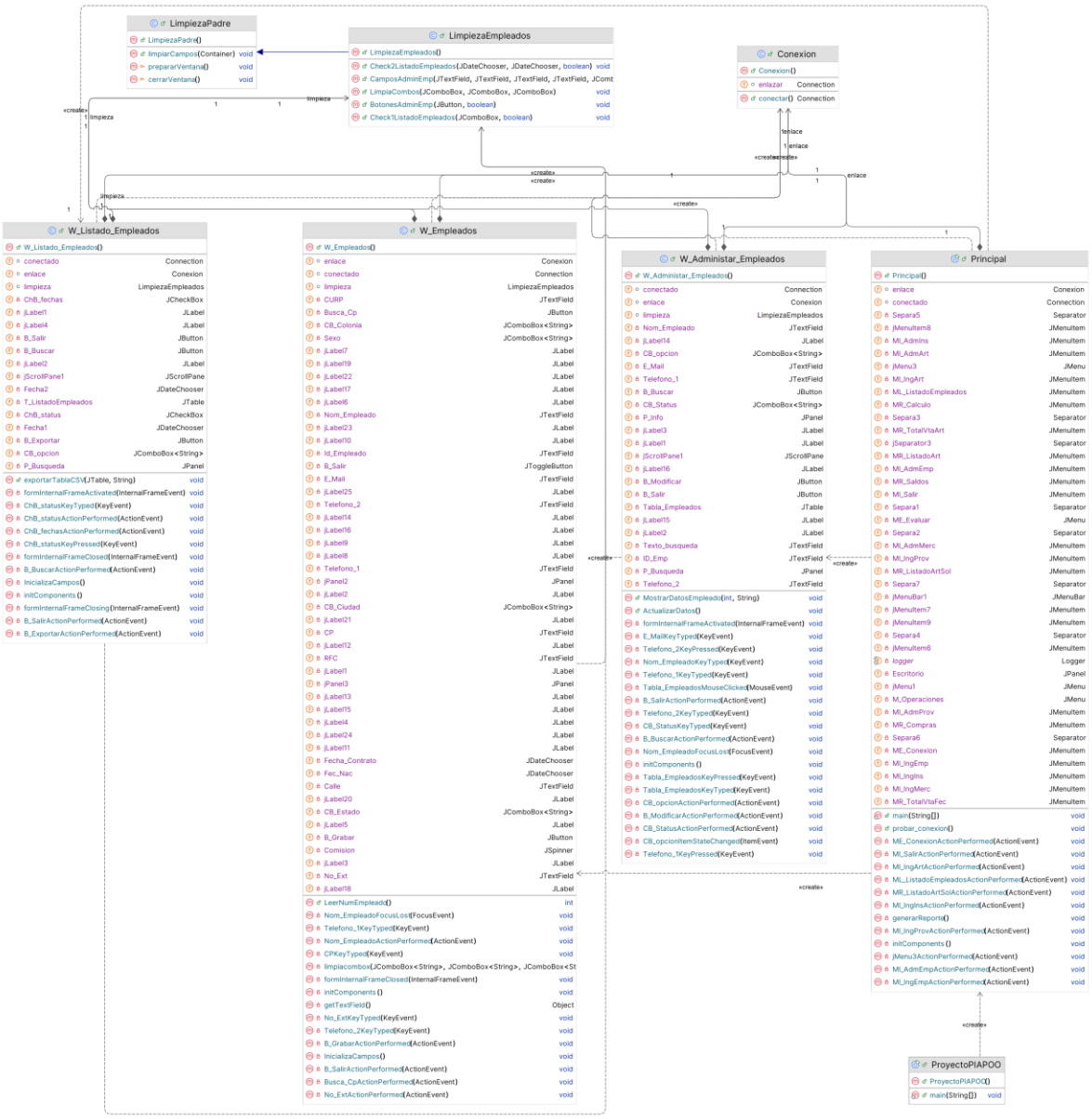
Esta estructura jerárquica garantiza la reutilización de código y promueve la consistencia entre las diferentes ventanas del sistema. Al concentrar las rutinas de validación y limpieza en una clase independiente, se evita la duplicación de lógica y se facilita la actualización o extensión futura del sistema.

En conjunto, el diseño técnico refleja una arquitectura multicapa, en la que cada componente cumple un rol claramente definido:

- **Capa de presentación:** compuesta por las ventanas (W_...) y el menú principal (Principal), encargadas de la interacción con el usuario.
- **Capa lógica o de negocio:** integrada por las clases de limpieza y validación, que manejan la coherencia interna y las reglas de funcionamiento del sistema.
- **Capa de datos:** representada por la clase Conexión, que abstrae la comunicación con la base de datos MySQL y centraliza todas las operaciones SQL.

El uso del paradigma orientado a objetos, la herencia, la modularidad y la comunicación mediante instancias controladas permiten que el Sistema Administrativo sea fácil de mantener, escalar y depurar. El diagrama UML evidencia esta arquitectura al mostrar la interrelación entre clases, la jerarquía funcional y el flujo de dependencias, reflejando un diseño bien estructurado y adaptable a futuras ampliaciones, como nuevos módulos administrativos o funcionalidades adicionales.

DIAGRAMA UML COMPLETO



ESTRUCTURA GENERAL DE LAS TABLAS

1. **Tabla empleados**

   Esta tabla almacena la información principal del personal registrado en la empresa. Contiene campos como el identificador del empleado (Id_Empleado), nombre, dirección, teléfonos, RFC, CURP, correo electrónico, comisión y estatus laboral. Además, incluye atributos relacionados con su contrato y sexo. Es la tabla de mayor relevancia para el módulo del **Sistema Administrativo**, ya que provee los datos que el sistema consulta, inserta o actualiza directamente a través de las interfaces gráficas.

   La estructura está optimizada para operaciones de búsqueda y actualización, garantizando un acceso eficiente a los registros del personal.

2. **Tabla artículos**

   Contiene el catálogo de productos o bienes manejados por la empresa. Cada artículo está identificado por su Id_Articulo y asociado a un insumo mediante Id_Insumo_Art. Se incluyen además campos que indican temporada de siembra y cosecha, inventario disponible y precio de venta. Su propósito principal es servir de referencia para el módulo de ventas y control de inventario.

3. **Tabla insumos**

   Define los materiales o recursos necesarios para la producción o mantenimiento de artículos. Incluye campos como Id_Insumo, Nom_Insumo, inventario disponible, punto de reorden y precio de compra. Cada insumo está vinculado con un proveedor (Id_Proveedor_Ins), lo que permite rastrear la fuente de suministro.

4. **Tabla proveedores**

   Almacena la información de los proveedores asociados a los insumos. Incluye datos de identificación, dirección, RFC, teléfono, correo electrónico y porcentaje de descuento aplicado. Esta tabla es fundamental para los módulos de compras y abastecimiento, ya que se enlaza con las tablas compra_encab y compra_detalle.

5. **Tabla compra_encab** y **compra_detalle**

   Estas dos tablas conforman la estructura de registro de compras.

   - **compra_encab** guarda los encabezados de las órdenes de compra, con información general como proveedor, subtotal, IVA, total y estado de la compra (pendiente o surtido).

   - **compra_detalle** complementa este registro con los productos adquiridos, sus cantidades y precios unitarios.
     Juntas, mantienen la trazabilidad de los movimientos de inventario y costos de adquisición.

6. **Tabla venta_encab** y **venta_detalle**

   Funcionan de manera análoga a las tablas de compra, pero enfocadas en el proceso de venta.

   - **venta_encab** registra la información general de cada transacción: el mercado, el empleado que realizó la venta, los totales, IVA, descuentos y comisiones.

   - **venta_detalle** almacena los artículos vendidos, las cantidades y los precios asociados.
     Estas tablas permiten generar reportes de ventas, calcular comisiones y actualizar inventarios en función de los movimientos comerciales.

7. **Tabla mercados**

   Contiene los datos de los mercados o puntos de venta. Almacena la dirección, contactos y descuentos aplicables a cada cliente o sucursal. Su función principal es servir como referencia para las ventas y el cálculo de descuentos por mercado.

8. **Tabla saldo_empresa**

   Registra los saldos diarios de la empresa, funcionando como una bitácora financiera. Cada registro corresponde a una fecha específica y un saldo en caja chica. Este control es esencial para la gestión contable y el seguimiento de movimientos financieros internos.

9. **Tabla facturas**

   Representa las facturas generadas en cada transacción de venta, vinculadas con el identificador de venta (Id_Venta_Fact). Su función es mantener la trazabilidad fiscal y documental del proceso comercial.

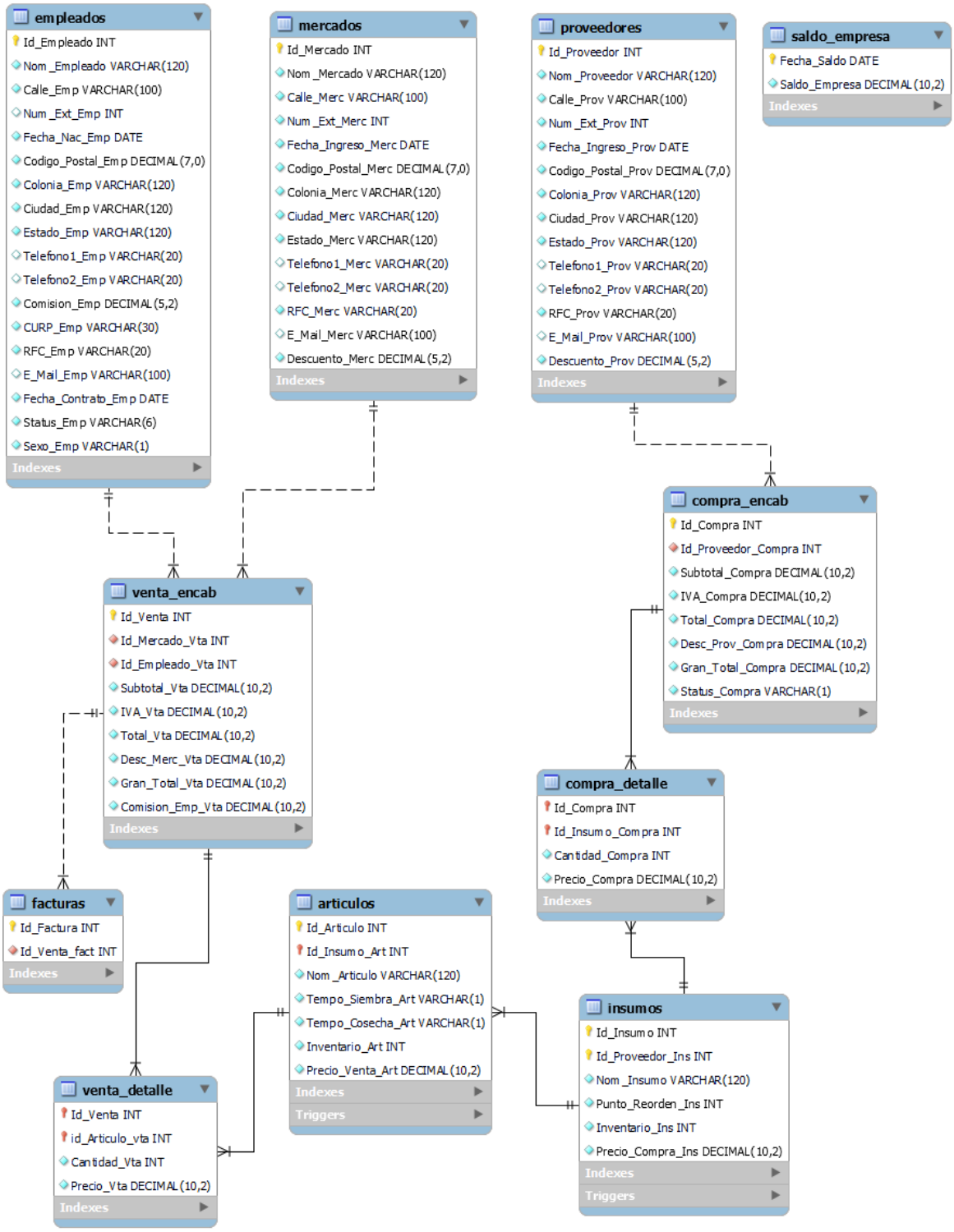
Características generales del diseño

El diseño de la base de datos **bd_inventario** sigue una **estructura relacional normalizada** y está optimizado para su uso en entornos empresariales pequeños y medianos. Se emplea el motor **InnoDB**, que proporciona soporte para transacciones, integridad referencial y bloqueo a nivel de fila, garantizando seguridad y consistencia en entornos multiusuario.

Todos los campos fueron definidos con tipos de datos adecuados (INT, VARCHAR, DECIMAL, DATE), utilizando el conjunto de caracteres **utf8mb4** para compatibilidad total con caracteres acentuados y símbolos propios del idioma español. Además, los comentarios incluidos en el esquema facilitan la comprensión de los valores codificados en ciertos campos (por ejemplo, estatus “A” o “B”, temporadas o estados de venta).

La base de datos fue concebida con **capacidad de expansión**, permitiendo la incorporación futura de módulos adicionales sin alterar la estructura principal. Esto la convierte en una base sólida para el crecimiento del **Sistema Administrativo**, posibilitando su evolución hacia un sistema integral de gestión empresarial.

DIAGRAMA ENTIDAD RELACIÓN



SCRIPT DE BD (SQL SCRIPT)

- **Configuración inicial del entorno SQL**

```sql
-- MySQL Script generated by MySQL Workbench
-- Sat Oct 11 14:33:47 2025
-- Model: New Model    Version: 1.0
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- -----------------------------------------------------
```

- **Creación del esquema 'bd_inventario'**

  Se crea el scheme en el que se almacenarán todas las tablas utilizadas dentro del Sistema Administrativo.

```sql
-- Schema mydb
-- -----------------------------------------------------
-- -----------------------------------------------------
-- Schema bd_inventario
-- -----------------------------------------------------
DROP SCHEMA IF EXISTS `bd_inventario` ;

-- -----------------------------------------------------
-- Schema bd_inventario
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS `bd_inventario` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci ;
USE `bd_inventario` ;
```

  A continuación se crean y asignan todas las tablas que manejará el Sistema Administrativo, cada una incluyendo sus respectivas columnas y primary keys.

- **Tabla 'artículos'**

```sql
-- -----------------------------------------------------
-- Table `bd_inventario`.`articulos`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`articulos` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`articulos` (
  `Id_Articulo` INT NOT NULL,
  `Id_Insumo_Art` INT NOT NULL,
  `Nom_Articulo` VARCHAR(120) NOT NULL,
  `Tempo_Siembra_Art` VARCHAR(1) NOT NULL COMMENT 'P = Primavera\\\nV = Verano\\\nO = Otoño\\\nI = Invierno\\\nT = Todas',
  `Tempo_Cosecha_Art` VARCHAR(1) NOT NULL COMMENT 'P = Primavera\\\nV = Verano\\\nO = Otoño\\\nI = Invierno\\\nT = Todas',
  `Inventario_Art` INT NOT NULL,
  `Precio_Venta_Art` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  PRIMARY KEY (`Id_Articulo`, `Id_Insumo_Art`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tablas de Articulos';
```

- **Tabla 'compra_detalle'**

```sql
-- -----------------------------------------------------
-- Table `bd_inventario`.`compra_detalle`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`compra_detalle` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`compra_detalle` (
  `Id_Compra` INT NOT NULL,
  `Id_Insumo_Compra` INT NOT NULL,
  `Cantidad_Compra` INT NOT NULL DEFAULT '0',
  `Precio_Compra` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  PRIMARY KEY (`Id_Compra`, `Id_Insumo_Compra`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla de la compra de insumos';
```

- **Tabla ‘compra_encab’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`compra_encab`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`compra_encab` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`compra_encab` (
  `Id_Compra` INT NOT NULL,
  `Id_Proveedor_Compra` INT NOT NULL,
  `Subtotal_Compra` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `IVA_Compra` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Total_Compra` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Desc_Prov_Compra` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Gran_Total_Compra` DECIMAL(10,2) NOT NULL,
  `Status_Compra` VARCHAR(1) NOT NULL DEFAULT 'P' COMMENT 'P = Pendiente\\nS = Surtido',
  PRIMARY KEY (`Id_Compra`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla para generar orden de compra de los insumos';
```

- **Tabla ‘empleados’**

```
-- Table `bd_inventario`.`empleados`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`empleados` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`empleados` (
  `Id_Empleado` INT NOT NULL,
  `Nom_Empleado` VARCHAR(120) NOT NULL,
  `Calle_Emp` VARCHAR(100) NOT NULL,
  `Num_Ext_Emp` INT NULL DEFAULT NULL,
  `Fecha_Nac_Emp` DATE NOT NULL,
  `Codigo_Postal_Emp` DECIMAL(7,0) NOT NULL,
  `Colonia_Emp` VARCHAR(120) NOT NULL,
  `Ciudad_Emp` VARCHAR(120) NOT NULL,
  `Estado_Emp` VARCHAR(120) NOT NULL,
  `Telefono1_Emp` VARCHAR(20) NULL DEFAULT NULL,
  `Telefono2_Emp` VARCHAR(20) NULL DEFAULT NULL,
  `Comision_Emp` DECIMAL(5,2) NOT NULL,
  `CURP_Emp` VARCHAR(30) NOT NULL,
  `RFC_Emp` VARCHAR(20) NOT NULL,
  `E_Mail_Emp` VARCHAR(100) NULL DEFAULT NULL,
  `Fecha_Contrato_Emp` DATE NOT NULL,
  `Status_Emp` VARCHAR(6) NOT NULL DEFAULT 'A' COMMENT 'A = Activo\\\\\\\\nB = Baja',
  `Sexo_Emp` VARCHAR(1) NOT NULL COMMENT 'M = Masculino\\\\nF = Femenino',
  PRIMARY KEY (`Id_Empleado`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla de Empleados Vendedores por Comision';
```

- **Tabla ‘facturas’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`facturas`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`facturas` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`facturas` (
  `Id_Factura` INT NOT NULL,
  `Id_Venta_fact` INT NOT NULL,
  PRIMARY KEY (`Id_Factura`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla de Facturas Generadas';
```

- **Tabla ‘insumos’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`insumos`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`insumos` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`insumos` (
  `Id_Insumo` INT NOT NULL,
  `Id_Proveedor_Ins` INT NOT NULL,
  `Nom_Insumo` VARCHAR(120) NOT NULL,
  `Punto_Reorden_Ins` INT NOT NULL DEFAULT '0',
  `Inventario_Ins` INT NOT NULL DEFAULT '0',
  `Precio_Compra_Ins` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`Id_Insumo`, `Id_Proveedor_Ins`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla de Insumos(Materiales para Producir un bien o producto)';
```

- **Tabla ‘mercados’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`mercados`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`mercados` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`mercados` (
  `Id_Mercado` INT NOT NULL,
  `Nom_Mercado` VARCHAR(120) NOT NULL,
  `Calle_Merc` VARCHAR(100) NOT NULL,
  `Num_Ext_Merc` INT NOT NULL,
  `Fecha_Ingreso_Merc` DATE NOT NULL,
  `Codigo_Postal_Merc` DECIMAL(7,0) NOT NULL,
  `Colonia_Merc` VARCHAR(120) NOT NULL,
  `Ciudad_Merc` VARCHAR(120) NOT NULL,
  `Estado_Merc` VARCHAR(120) NOT NULL,
  `Telefono1_Merc` VARCHAR(20) NULL DEFAULT NULL,
  `Telefono2_Merc` VARCHAR(20) NULL DEFAULT NULL,
  `RFC_Merc` VARCHAR(20) NOT NULL,
  `E_Mail_Merc` VARCHAR(100) NULL DEFAULT NULL,
  `Descuento_Merc` DECIMAL(5,2) NOT NULL DEFAULT '0.00' COMMENT 'Min 0% max 50%',
  PRIMARY KEY (`Id_Mercado`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;
```

- **Tabla ‘proveedores’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`proveedores`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`proveedores` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`proveedores` (
  `Id_Proveedor` INT NOT NULL,
  `Nom_Proveedor` VARCHAR(120) NOT NULL,
  `Calle_Prov` VARCHAR(100) NOT NULL,
  `Num_Ext_Prov` INT NOT NULL,
  `Fecha_Ingreso_Prov` DATE NOT NULL,
  `Codigo_Postal_Prov` DECIMAL(7,0) NOT NULL,
  `Colonia_Prov` VARCHAR(120) NOT NULL,
  `Ciudad_Prov` VARCHAR(120) NOT NULL,
  `Estado_Prov` VARCHAR(120) NOT NULL,
  `Telefono1_Prov` VARCHAR(20) NULL DEFAULT NULL,
  `Telefono2_Prov` VARCHAR(20) NULL DEFAULT NULL,
  `RFC_Prov` VARCHAR(20) NOT NULL,
  `E_Mail_Prov` VARCHAR(100) NULL DEFAULT NULL,
  `Descuento_Prov` DECIMAL(5,2) NOT NULL DEFAULT '0.00' COMMENT 'Min 0% max 50%',
  PRIMARY KEY (`Id_Proveedor`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;
```

- **Tabla ‘saldo_empresa’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`saldo_empresa`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`saldo_empresa` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`saldo_empresa` (
  `Fecha_Saldo` DATE NOT NULL,
  `Saldo_Empresa` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  PRIMARY KEY (`Fecha_Saldo`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Saldo Diario con que empieza en Caja Chica, Si son Negativos se cobra a Cajera';
```

- **Tabla ‘venta_detalle’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`venta_detalle`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`venta_detalle` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`venta_detalle` (
  `Id_Venta` INT NOT NULL,
  `id_Articulo_vta` INT NOT NULL,
  `Cantidad_Vta` INT NOT NULL DEFAULT '0',
  `Precio_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  PRIMARY KEY (`Id_Venta`, `id_Articulo_vta`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla donde se encuentran los articulos vendidos';
```

- **Tabla ‘venta_encab’**

```
-- -----------------------------------------------------
-- Table `bd_inventario`.`venta_encab`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `bd_inventario`.`venta_encab` ;

CREATE TABLE IF NOT EXISTS `bd_inventario`.`venta_encab` (
  `Id_Venta` INT NOT NULL,
  `Id_Mercado_Vta` INT NOT NULL,
  `Id_Empleado_Vta` INT NOT NULL,
  `Subtotal_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `IVA_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Total_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Desc_Merc_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Gran_Total_Vta` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `Comision_Emp_Vta` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`Id_Venta`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci
COMMENT = 'Tabla de Encabezado de Ventas';
```

- **Procedure Conteo_de_ciudades**

```
-- -----------------------------------------------------
-- procedure Conteo_de_ciudades
-- -----------------------------------------------------

USE `bd_inventario`;
DROP procedure IF EXISTS `bd_inventario`.`Conteo_de_ciudades`;

DELIMITER $$
USE `bd_inventario`$$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Conteo_de_ciudades`( IN var_estado INT, OUT conteo_ciudades INT)
BEGIN
    SELECT count(*) into conteo_ciudades FROM cuidades where Id_Estado = var_estado ;
END$$

DELIMITER ;
```

El procedimiento almacenado tiene la función de **contar el número total de ciudades asociadas a un estado específico**, según su identificador. Recibe como parámetro de entrada (IN var_estado) el identificador del estado que se desea analizar, y devuelve mediante un parámetro de salida (OUT conteo_ciudades) el resultado del conteo realizado en la tabla cuidades. Internamente, ejecuta una instrucción SELECT COUNT(*) que contabiliza cuántos registros existen en dicha tabla donde el campo Id_Estado coincide con el valor proporcionado. Su propósito principal es **proporcionar una forma rápida y automatizada de obtener información estadística** sobre la distribución de ciudades por estado, lo que puede resultar útil para reportes administrativos, análisis geográficos o validaciones de integridad dentro del sistema.

- **Procedure Reporte_Listado_Articulos**

```
USE `bd_inventario`;
DROP procedure IF EXISTS `bd_inventario`.`Reporte_Listado_Articulos`;

DELIMITER $$
USE `bd_inventario`$$
CREATE DEFINER=`root`@`localhost` PROCEDURE `Reporte_Listado_Articulos`()
BEGIN
SELECT A.id_articulo as ID_ART,
    A.nom_articulo as Nombre_Art,
    A.Precio_Venta_Art as Precio_Venta,
    I.id_insumo as Num_Insumo,
    I.Nom_Insumo as Nombre_Ins,
    I.Punto_Reorden_Ins as Reorden,
    I.Inventario_Ins as Inventario,
    P.Nom_Proveedor as Proveedor,
    I.Precio_Compra_Ins as Precio_Ins
    FROM articulos A, Insumos I, Proveedores P
    WHERE I.id_insumo = A.Id_insumo_Art AND
    P.Id_Proveedor = I.Id_Proveedor_Ins
    ORDER BY A.Id_Articulo ASC, I.id_insumo ASC;
END$$

DELIMITER ;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
USE `bd_inventario`;
DELIMITER ;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
USE `bd_inventario`;

DELIMITER $$

USE `bd_inventario`$$
DROP TRIGGER IF EXISTS `bd_inventario`.`articulos_BEFORE_INSERT` $$
USE `bd_inventario`$$
CREATE
DEFINER=`root`@`localhost`
TRIGGER `bd_inventario`.`articulos_BEFORE_INSERT`
BEFORE INSERT ON `bd_inventario`.`articulos`
FOR EACH ROW
BEGIN
/*Checa que el Articulo No tenga mas de 10 Insumos Asignados*/
    IF (    SELECT COUNT(*)
                FROM Articulos
                WHERE Id_Articulo = new.id_Articulo
                GROUP BY Id_Articulo)>10 THEN
        SIGNAL SQLSTATE "45000"
        SET MESSAGE_TEXT = "No se puede Agregar mas de 10 Insumos por Articulo";
    END IF;
END$$
```

```mysql
USE `bd_inventario`$$
DROP TRIGGER IF EXISTS `bd_inventario`.`insumos_BEFORE_INSERT` $$
USE `bd_inventario`$$
CREATE
DEFINER=`root`@`localhost`
TRIGGER `bd_inventario`.`insumos_BEFORE_INSERT`
BEFORE INSERT ON `bd_inventario`.`insumos`
FOR EACH ROW
BEGIN
    /*Checa que el Insumo No tenga mas de 10 Proveedores Asignados*/
    IF (    SELECT COUNT(*)>10
                FROM Insumos
                WHERE Id_insumo = new.id_insumo
                GROUP BY Id_insumo) >10 THEN
        SIGNAL SQLSTATE "45000"
        SET MESSAGE_TEXT = "No se puede Agregar mas de 10 Proveedores por Insumo";
    END IF;
END$$


DELIMITER ;
```

El procedimiento genera un **reporte consolidado** que cruza el catálogo de artículos con sus insumos y el proveedor correspondiente. Al ejecutarse, realiza una consulta sobre las tablas articulos (A), insumos (I) y proveedores (P) usando condiciones de unión implícitas (I.id_insumo = A.Id_insumo_Art y P.Id_Proveedor = I.Id_Proveedor_Ins) y devuelve, por cada artículo, su identificador y nombre (ID_ART, Nombre_Art), el **precio de venta** (Precio_Venta), el **número y nombre del insumo** asociado (Num_Insumo, Nombre_Ins), el **punto de reorden** e **inventario** del insumo (Reorden, Inventario), el **proveedor** (Proveedor) y el **precio de compra** del insumo (Precio_Ins). El resultado se ordena ascendentemente por A.Id_Articulo y luego por I.id_insumo, facilitando su uso en listados o reportes operativos. No recibe parámetros ni modifica datos: su objetivo es exclusivamente de consulta, estandarizando en una sola salida información clave para abastecimiento, control de inventarios y análisis de márgenes (relación precio de compra vs. precio de venta).

5.- INSTALACIÓN TÉCNICA

1. Descarga del paquete de instalación

1. Descargue el **paquete de instalación** del **Sistema Administrativo**, el cual contiene el proyecto Java completo, junto con los archivos fuente, recursos gráficos y la carpeta con el script SQL correspondiente.

2. Guarde el contenido del paquete en una ubicación accesible de su equipo (por ejemplo, C:\SistemaAdministrativo o Documentos\Sistema_Administrativo).

---

2. Creación de la base de datos

1. Abra **MySQL Workbench** o cualquier administrador de bases de datos compatible con MySQL.

2. Cree una **nueva conexión** y asígnele el nombre:

   BD_Inventario

3. Dentro de la conexión creada, abra una nueva pestaña de consultas SQL.

4. Copie el contenido completo del archivo **SQLScript_BD_Inventario.sql** que se incluye en el paquete del sistema.

5. Pegue el script en la pestaña SQL y ejecútelo.
   Esto creará automáticamente el esquema **bd_inventario**, junto con todas sus tablas, procedimientos almacenados y triggers necesarios para el funcionamiento del sistema.

6. Una vez ejecutado correctamente, verifique en el panel lateral que la base de datos **bd_inventario** haya sido creada y que las tablas (como empleados, articulos, insumos, proveedores, etc.) estén visibles.

Nota: En caso de que el script arroje errores por permisos o restricciones, asegúrese de ejecutar MySQL Workbench con privilegios de administrador.

3. Configuración del entorno de desarrollo

1. Abra el entorno de desarrollo **Apache NetBeans** o **IntelliJ IDEA**.

2. Seleccione la opción **"Abrir proyecto"** y busque la carpeta principal del programa que descargó en el paso 2.

3. Una vez abierto el proyecto, asegúrese de que todas las dependencias necesarias estén cargadas correctamente (especialmente el conector MySQL).

   - Si el IDE solicita agregar el archivo **mysql-connector-java-x.x.x.jar**, adjúntelo manualmente en la sección de **Librerías del proyecto**.

- Este conector debe ubicarse generalmente en la ruta:
- C:\Program Files\MySQL\Connector J 8.0\mysql-connector-java-8.0.xx.jar

4. Configuración del archivo de conexión

1. Dentro del proyecto, localice el archivo **Conexion.java**, generalmente ubicado en el paquete principal del sistema.
2. Abra el archivo y diríjase a la **línea 15**, donde encontrará la siguiente instrucción:

   conexion = DriverManager.getConnection("jdbc:mysql://localhost/bd_inventario", "root", "root");

3. El segundo valor "root" corresponde al **usuario** de la base de datos MySQL (por defecto suele ser “root”), y el tercer parámetro "root" representa la **contraseña**.
4. Sustituya el segundo "root" por la **contraseña que usted haya configurado para su servidor MySQL**.
   Por ejemplo, si su contraseña es admin123, la línea quedaría así:

   conexion = DriverManager.getConnection("jdbc:mysql://localhost/bd_inventario", "root", "admin123");

5. Guarde los cambios realizados en el archivo y cierre el editor.

Importante: No modifique el nombre de la base de datos (bd_inventario) dentro de la cadena de conexión, ya que el sistema fue diseñado para trabajar específicamente con ese esquema.

5. Ejecución del sistema

1. Una vez configurada la conexión, busque dentro del proyecto el archivo **main** o **ProyectoPAPOO.java**, el cual contiene el método main(String[] args).
2. Ejecútelo desde su IDE seleccionando la opción **“Run”** o presionando **Shift + F6** (en NetBeans).
3. Espere a que el programa compile y cargue la interfaz principal del sistema.
4. Al iniciar, se abrirá la ventana principal con el menú de opciones.
5. Para verificar que la conexión con la base de datos fue exitosa, seleccione el botón **“Evaluar”** y posteriormente el submenú **“Conexión”**.
   Si la conexión fue configurada correctamente, deberá aparecer un mensaje de confirmación similar al siguiente:
6. Conexión exitosa con la base de datos BD_Inventario.

En caso contrario, revise nuevamente la contraseña y la disponibilidad del servidor MySQL.

6. Verificación y prueba de funcionamiento

Una vez instalado y configurado, se recomienda realizar las siguientes pruebas iniciales:

- Registrar un nuevo empleado desde la interfaz de administración y comprobar que se almacena en la tabla empleados.
- Consultar el listado de empleados y verificar que la información se recupere correctamente.
- Cerrar y volver a abrir el programa para confirmar la persistencia de la información.

Estas pruebas básicas garantizan que el sistema se ha instalado y configurado correctamente, y que la comunicación entre la aplicación Java y la base de datos **bd_inventario** funciona de forma estable.

7. Consideraciones adicionales

- En caso de reinstalar el sistema o migrarlo a otro equipo, asegúrese de **respaldar la base de datos** utilizando la opción *Export Database* desde MySQL Workbench.
- Si desea cambiar la contraseña del usuario de MySQL, actualice también la cadena de conexión en Conexion.java.
- Para mantener el correcto funcionamiento, no elimine ni modifique las tablas o procedimientos almacenados del script original, ya que el programa depende de ellos para operar correctamente.

6.- DETALLE DE MÓDULOS Y FUNCIONES

A continuación se mostrarán ejemplos de código de partes importantes del programa que sean ejemplos claros de acciones cotidianas del código.

MÉTODO LEERNUMEROEMPLEADO

```
Conexion.java ×  ProyectoPIAPOO.java ×  Principal.java ×  W_Empleados.java ×  W_Listado_Empleados.java ×
urce  Design  History

    }

    public int LeerNumEmpleado() {
     int folioInt = 0; // valor por defecto

     String SQLText = "SELECT id_empleado + 1 FROM empleados ORDER BY id_empleado DESC LIMIT 1";
     String folioStr ;

     try (Statement Leer = conectado.createStatement();
     ResultSet Resultado = Leer.executeQuery(SQLText))
     {
         if (Resultado.next()) {
             // Sí hay datos en la consulta
             folioStr = Resultado.getString(1);
         } else {
             // No hay datos → folio por defecto = 100
             folioStr = "100";
         }

         try
         {
             folioInt = Integer.parseInt(folioStr);
         } catch (NumberFormatException e) {
             JOptionPane.showMessageDialog(null, e + ". Error al convertir folio a entero.", "AVISO", JOptionPane.WARNING_M
         }

     } catch (SQLException e)
     {
         JOptionPane.showMessageDialog(null, e + ". Error en la consulta SQL.", "AVISO", JOptionPane.WARNING_MESSAGE);
     }
```

Este método es el responsable de asignar el I.D de cada empleado siendo capturado en el registro. Como puede ver, su funcionamiento depende de la consulta de los registros concurrentes contenidos dentro de la base de datos al momento. Esto es para evitar repetir números de empleados y posiblemente reemplazar registros o cometer otros errores dentro del registro de empleados.

METODO EXPORTAR TABLA

Conexion.java × ProyectoPIAPOO.java × Principal.java × W_Empleados.java × W_Listado_Empleados.java ×

Source Design History

```
346
347 public void exportarTablaCSV(JTable tabla, String rutaArchivo) {
348     try {
349         TableModel modelo = tabla.getModel();
            FileWriter archivo = new FileWriter(rutaArchivo);
351         PrintWriter pw = new PrintWriter(archivo);
352
353         // Escribir encabezados
354         for (int i = 0; i < modelo.getColumnCount(); i++) {
355             pw.print(modelo.getColumnName(i));
356             if (i != modelo.getColumnCount() - 1) {
357                 pw.print(",");
358             }
359         }
360         pw.println();
361
362         // Escribir filas
363         for (int i = 0; i < modelo.getRowCount(); i++) {
364             for (int j = 0; j < modelo.getColumnCount(); j++) {
365                 Object valor = modelo.getValueAt(i, j);
366                 // Escapar comas en los datos
367                 String celda = valor != null ? valor.toString().replace(",", " ") : "";
368                 pw.print(celda);
369                 if (j != modelo.getColumnCount() - 1) {
370                     pw.print(",");
371                 }
372             }
373             pw.println();
374         }
375
376         pw.close();
377         archivo.close();
```

com.mycompany.proyectopiapoo.W_Listado_Empleados > B_BuscarActionPerformed > SQLWhere >

El método de exportación de registros es una buena muestra de la utilización de métodos de escritura y lectura activa de archivos concurrentes y de datos de la base de datos. Esto se hace utilizando instancias de ciclos for.

LIMPIAR CAMPOS

```java
public class LimpiezaPadre extends JFrame {

    // Método recursivo para limpiar todos los campos dentro de un contenedor
    public void limpiarCampos(Container container) {
        for (Component c : container.getComponents()) {

            // Limpiar JTextField
            if (c instanceof JTextField) {
                ((JTextField) c).setText("");

            // Limpiar JTextArea
            } else if (c instanceof JTextArea) {
                ((JTextArea) c).setText("");

            // Limpiar JCheckBox
            } else if (c instanceof JCheckBox) {
                ((JCheckBox) c).setSelected(false);

            // Limpiar JTable
            } else if (c instanceof JTable) {
                ((JTable) c).clearSelection();

            // Limpiar JDateChooser (campo de fecha)
            } else if (c instanceof JDateChooser) {
                ((JDateChooser) c).setDate(null);

            // Si es un contenedor (por ejemplo, JPanel), llamar recursivamente
            } else if (c instanceof Container) {
                limpiarCampos((Container) c);
            }
        }
    }
```

Como último ejemplo, podemos ver el método de LimpiarCampos que se suele llamar cada que se cierra o se reinicia una ventana que contenga campos que hayan sido rellenados. Como podemos apreciar, hay ciertas instancias que se llaman de forma recursiva para asegurar la limpieza de datos adecuadamente.

7.- CONSIDERACIONES FINALES

El presente **Manual Técnico del Sistema Administrativo de Ventas** ha sido elaborado con el propósito de servir como una guía de referencia completa para desarrolladores, técnicos, administradores de sistemas y demás personal involucrado en el mantenimiento, comprensión o ampliación del software. A lo largo de este documento se han descrito detalladamente los componentes técnicos, la estructura del código, la arquitectura del sistema, la base de datos y los procedimientos necesarios para garantizar el funcionamiento correcto y estable de la aplicación.

Se espera que la información aquí contenida sea de utilidad tanto para **fines académicos, de soporte técnico o de implementación práctica**, proporcionando las bases necesarias para comprender el diseño lógico y funcional del sistema. Asimismo, este manual puede servir como punto de partida para futuras mejoras o versiones del programa, ya que documenta los principios de diseño y las tecnologías empleadas en su desarrollo.

El objetivo final de este documento es **facilitar la continuidad técnica del proyecto**, promoviendo buenas prácticas de documentación y asegurando que cualquier persona que requiera trabajar con el sistema —ya sea para mantenimiento, actualización o estudio— cuente con una fuente confiable y clara de información.

Con ello, se concluye este manual con la convicción de que su contenido será de apoyo real para quienes busquen **entender, mantener o evolucionar** el **Sistema Administrativo de Ventas**, contribuyendo así a su estabilidad, escalabilidad y correcto aprovechamiento dentro de cualquier entorno donde sea implementado.